



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

TAREA 3-Fase 3



Docente: Perla Marlene Viera Gonzalez

Integrantes del equipo:

Jacob Misael Rodriguez Morales 1907926

Matthew Rosas Lailosn 2132902

Jahir Guadalupe Salazar Esparza 1895234

Cruz Eduardo Patiño Zuñiga 2010046

Fecha: 13/04/2026

1. Resumen ejecutivo

El objetivo de este análisis fue realizar un examen forense integral a un binario educativo diseñado por el equipo para comportarse de manera sospechosa, aunque se mantiene 100% benigno. Para esto, seguimos la metodología de la fase 3 de la materia, pasando por el análisis estático profundo, ingeniería inversa y depuración dinámica en entornos aislados. Este proceso no solo nos sirvió para ver qué hace el código, sino para entender cómo el compilador transforma nuestras instrucciones de C++ en lenguaje ensamblador y cómo se reflejan estas en la pila y los registros del procesador durante la ejecución.

Durante la fase estática, identificamos firmas críticas en las secciones del binario, destacando la cadena única "MAGIC: edu-torres-malware-sim" y rutas que apuntan a la creación de archivos en directorios temporales, como C:\temp\dummyedu.txt. Al pasar al análisis dinámico con x64dbg, confirmamos que el programa utiliza técnicas comunes de evasión, como un retardo de tiempo mediante la función Sleep por un periodo de 5000 milisegundos (5 segundos), una táctica clásica para intentar engañar a las sandboxes automáticas. También pudimos observar en tiempo real cómo se preparan los parámetros en el *stack* antes de llamar a APIs críticas de Windows como WinExec para abrir la calculadora y CreateFileA para manipular el sistema de archivos.

Como conclusión, los hallazgos técnicos nos permitieron extraer Indicadores de Compromiso (IOCs) claros, lo que facilita la creación de medidas de detección efectivas. Recomendamos el despliegue de firmas YARA basadas en los patrones de texto encontrados y el monitoreo de llamadas a funciones de ejecución en rutas no comunes para fortalecer la postura de seguridad frente a este tipo de comportamientos.

3.Índice

TAREA 3-Fase 3	1
1. Resumen ejecutivo.....	2
4. Introducción	4
5. Metodología.....	4
5.1 Entorno de pruebas y configuración del laboratorio.....	4
5.2 Kit de herramientas y versiones.....	5
5.3 Pasos detallados del análisis	5
6. Análisis estático.....	8
6.1 Cadenas y recursos relevantes	8
6.2 Imports y llamadas críticas	8
6.3 Detección de ofuscación y packers	8
7. Análisis dinámico.....	8
7.1 Comportamiento en ejecución	9
7.2 Intercepción de llamadas y registros	9
7.3 Monitoreo de memoria y procesos.....	9
8. Hallazgos (IOCs).....	9
8.1 Identificadores del archivo	10
8.2 Rastros en el sistema (Host-Based Indicators)	10
8.3 Firmas de memoria y comportamiento	10
9. Diagramas de flujo y pseudocódigo.....	10
10. Recomendaciones	11
11. Conclusiones	12
12. Referencias y anexos.....	12
12.2 Anexos y registros técnicos.....	13

4. Introducción

Este ejercicio lo hicimos para probar cómo se analiza un archivo que, aunque no es peligroso, usa trucos típicos para pasar desapercibido. La idea fue programar un ejecutable en C++ que incluyera mañas sencillas, como quedarse en pausa un rato antes de hacer cualquier cosa. El objetivo real es aprender a detectar estas señales y entender cómo se mueve un programa de este tipo cuando intenta ocultar lo que hace.

En cuanto al alcance, nos metimos a fondo en tres niveles. Primero revisamos el archivo por fuera para ver qué traía escrito; luego usamos herramientas de ingeniería inversa para desarmar el código y entender su lógica; y al final lo pusimos a correr en una máquina con Windows para ver exactamente cómo se comporta en la memoria. Así pudimos ver el proceso completo, desde que el archivo está guardado hasta que empieza a interactuar con el sistema.

Eso sí, hay que dejar claro que el binario es totalmente inofensivo. Aunque simula acciones que podrían parecer una amenaza, como abrir procesos o escribir archivos en carpetas del sistema, no tiene ninguna función que pueda dañar la computadora. Todo lo que hace, como crear el archivo `dummyedu.txt`, está controlado y solo sirve para practicar las técnicas de análisis en un entorno seguro.

5. Metodología

5.1 Entorno de pruebas y configuración del laboratorio

Para analizar este binario de forma segura, montamos un laboratorio basado en la segmentación y el uso de máquinas virtuales, evitando cualquier riesgo para el sistema principal. El desarrollo del código se hizo en una máquina host con Arch Linux, pero todas las pruebas de ejecución y debugging las pasamos a una máquina virtual con Windows 10.

Usamos VirtualBox para gestionar la VM, configurando una red interna (Host-Only) para que el tráfico estuviera totalmente aislado y no hubiera conexiones externas no deseadas. Antes de correr el binario por primera vez, generamos un *snapshot* limpio del sistema; esto es clave porque nos permite regresar la máquina virtual a su estado original después de cada ejecución, asegurando que los restos de archivos creados o cambios en el registro no ensucien los siguientes análisis.

5.2 Kit de herramientas y versiones

El análisis lo dividimos usando un set de herramientas específico para cada nivel de profundidad, asegurándonos de que cubrieran tanto la parte estática como la dinámica:

- **Ingeniería Inversa:** Utilizamos **Ghidra v10.1.4** para desarmar el binario y tratar de reconstruir el código original en C a través de su decompilador. También usamos **Radare2 y Cutter** cuando necesitamos una vista más rápida de las funciones y para preparar el parcheo de instrucciones.
- **Análisis Dinámico:** El peso de la depuración cayó sobre **x64dbg**, que nos permitió pausar el programa justo antes de que llamara a las APIs del sistema y ver qué traían los registros en ese momento.
- **Análisis Estático y Clasificación:** Para extraer información rápida sin ejecutar nada, usamos **Python con la librería pefile**, que nos ayudó a mapear las cabeceras del ejecutable, además de **CAPA** para detectar automáticamente qué capacidades sospechosas tiene el archivo.

5.3 Pasos detallados del análisis

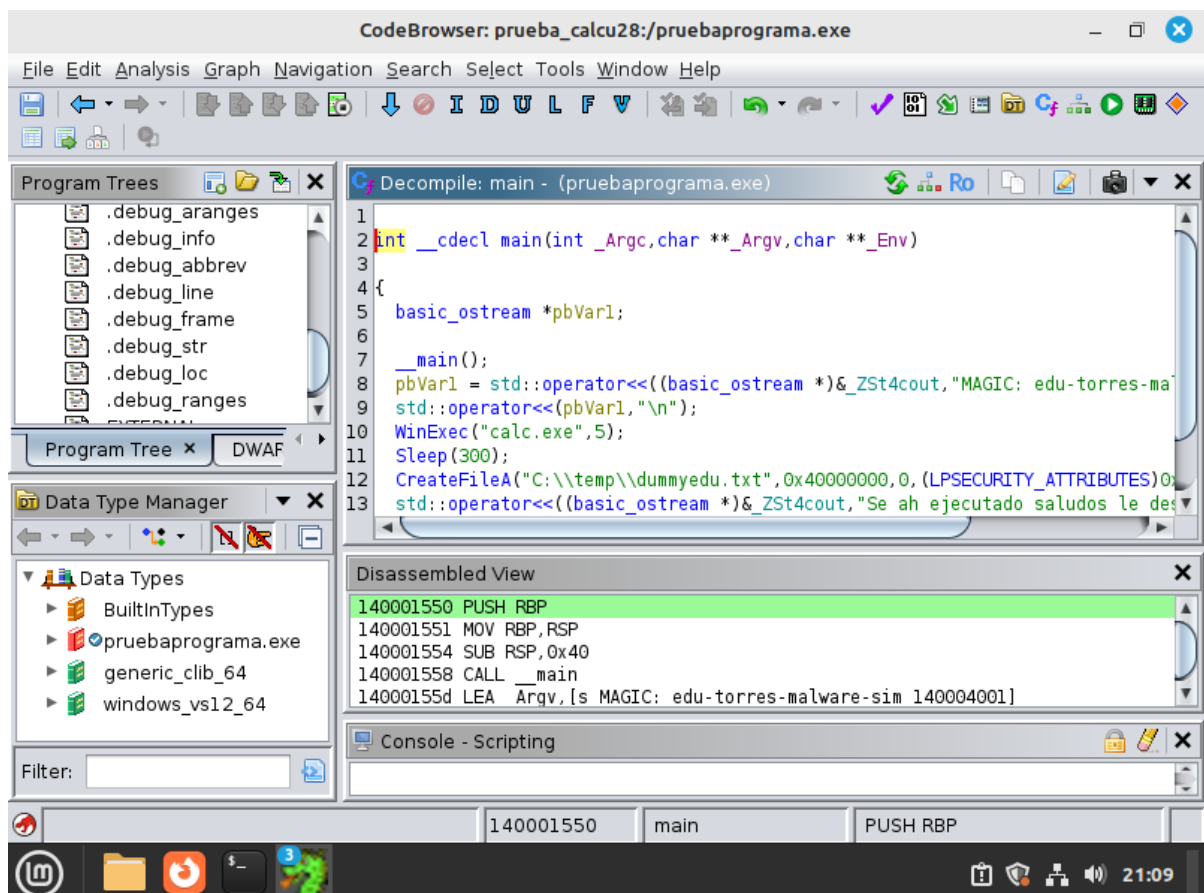
Fase 1: Análisis Estático Profundo El primer paso fue revisar el archivo sin darle "play" para no correr riesgos. Empezamos extrayendo todas las cadenas de texto (*strings*) con **rabin2** y un script de **pefile** para buscar rutas de archivos, mensajes ocultos o nombres de funciones que nos dieran una pista de qué iba a pasar. Después, mapeamos la tabla de importaciones para ver qué librerías de Windows estaba pidiendo el binario; ahí fue donde saltaron las alertas al ver funciones como **WinExec** y **CreateFileA**, que básicamente sirven para ejecutar cosas y mover archivos.

Fase 2: Ingeniería Inversa (Ghidra y Cutter) Con las sospechas del paso anterior, metimos el binario a Ghidra para desarmarlo. Localizamos la función **main** y analizamos el código en ensamblador para ver cómo estaba estructurada la lógica. Aquí identificamos el truco del retardo (*Sleep*) y cómo el programa preparaba los nombres de los archivos en la memoria antes de crearlos. También usamos Cutter para localizar instrucciones condicionales y probar qué pasaría si cambiábamos el flujo del programa, lo que nos sirvió para entender cómo se podría neutralizar una amenaza parcheando el código.

Fase 3: Debugging Dinámico (x64dbg) Esta fue la parte más movida, donde corrimos el binario paso a paso dentro de x64dbg. Pusimos puntos de interrupción (*breakpoints*) justo en las llamadas a las APIs de Windows que ya habíamos detectado. Al detener el programa, nos pusimos a revisar el *stack* y los registros para ver exactamente qué comandos le estaba mandando al sistema operativo, confirmando así que el programa sí abría la calculadora y escribía en la carpeta temporal.

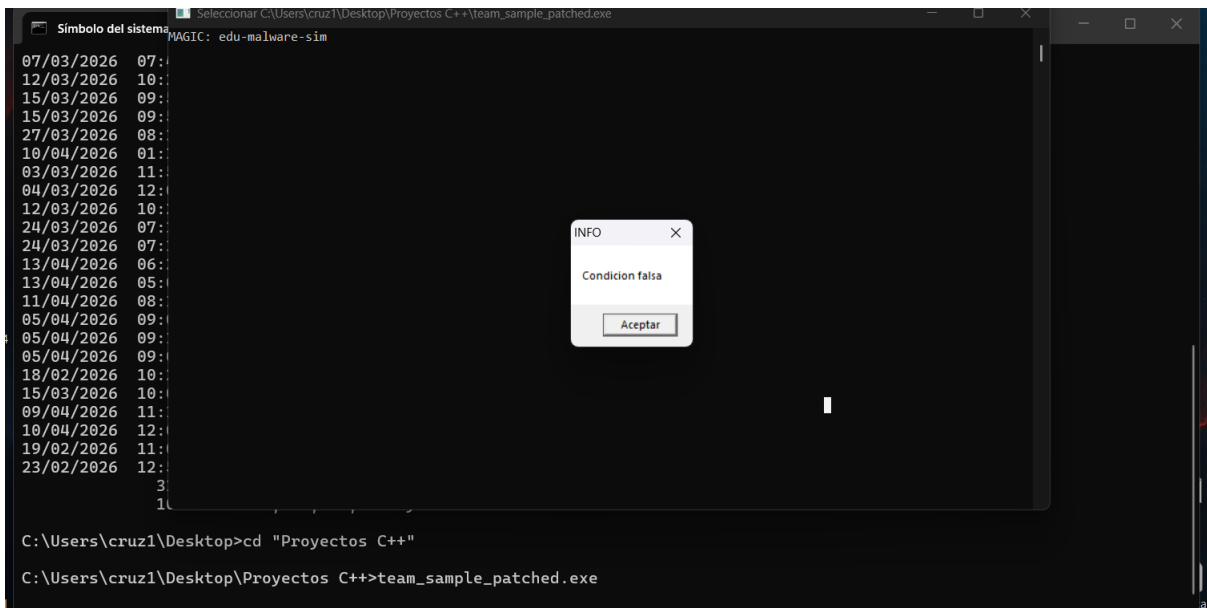
Fase 4: Clasificación y Detección Para cerrar, usamos CAPA para generar un reporte automático que clasificara todas las capacidades que encontramos. Con toda esta

información, redactamos una regla YARA personalizada que busca patrones específicos del binario, como la cadena "MAGIC", para que cualquier sistema de seguridad pueda detectar este archivo en el futuro sin necesidad de volverlo a analizar.



```
C:\Users\cruz1\Desktop\Proyectos C++>r2 -AA team_sample.exe
INFO: Second cons!
WARN: Skipping calculating actual checksum because too large file (1+ GiB)
WARN: Skipping calculating actual checksum because too large file (1+ GiB)
WARN: Relocs has not been applied. Please use '-e bin.relocs.apply=true' or '-e bin.cache=true' next time
WARN: Skipping calculating actual checksum because too large file (1+ GiB)
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze entrypoint (af@ entry0)
INFO: Analyze symbols (af@@@s)
INFO: Analyze all functions arguments/locals (afva@@F)
INFO: Analyze function calls (aac)
INFO: Analyze len bytes of instructions for references (aar)
INFO: Finding and parsing C++ vtables (avrr)
INFO: Analyzing methods (af @@ method.*)
INFO: Recovering local variables (afva@@@F)
INFO: Type matching analysis for all functions (aaft)
INFO: Propagate noreturn information (aanr)
INFO: Integrate dwarf function information
INFO: Scanning for strings constructed in code (/azs)
INFO: Finding function preludes (aap)
INFO: Enable types.constraint for experimental type propagation
INFO: Running plugin post-analysis hooks
-- No cookie is also a cookie
```

```
[0x004012e0]> aaa
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze entrypoint (af@ entry0)
INFO: Analyze symbols (af@@@s)
INFO: Analyze all functions arguments/locals (afva@@F)
INFO: Analyze function calls (aac)
INFO: Analyze len bytes of instructions for references (aar)
INFO: Finding and parsing C++ vtables (avrr)
INFO: Analyzing methods (af @@ method.*)
INFO: Recovering local variables (afva@@@F)
INFO: Type matching analysis for all functions (aajt)
INFO: Propagate noreturn information (aanr)
INFO: Integrate dwarf function information
INFO: Use -AA or aaaa to perform additional experimental analysis
```



6. Análisis estático

esta primera etapa nos dedicamos a desmenuzar el binario sin llegar a ejecutarlo para entender cómo está construido y qué pistas nos dejó el compilador.

6.1 Cadenas y recursos relevantes

Al pasar el comando `strings` y `rabin2 -zz`, lo primero que saltó fue la firma personalizada que le pusimos al código: `MAGIC: edu-torres-malware-sim`. Esta cadena es clave porque funciona como una marca de agua única para identificar el archivo en cualquier sistema. También encontramos rutas de archivos grabadas en el binario, específicamente `C:\temp\dummyedu.txt`, lo que ya nos daba una idea clara de dónde iba a intentar escribir información el programa. Además, el análisis con `pefile` nos confirmó que el binario fue compilado con GCC en un entorno Windows, manteniendo las secciones estándar como `.text` para el código y `.data` para las variables.

6.2 Imports y llamadas críticas

Revisamos la tabla de importaciones para ver con qué librerías del sistema se comunica el archivo. Notamos que jala funciones de `KERNEL32.dll` que son muy comunes en programas que manipulan el sistema:

- **WinExec:** Esta es la que usa para lanzar la calculadora (`calc.exe`).
- **CreateFileA:** La utiliza para generar el archivo de texto en la carpeta temporal.
- **Sleep:** Es la que provoca el retraso de 5 segundos que detectamos después en el análisis dinámico.

6.3 Detección de ofuscación y packers

Para estar seguros de que no había nada escondido, checamos si el binario estaba comprimido con algo como UPX. Al revisar la entropía de las secciones y buscar firmas de *packers* conocidos, confirmamos que el código está "limpio" y no tiene capas de ofuscación que intenten engañar al desensamblador. Esto nos facilitó mucho el trabajo en la siguiente fase de ingeniería inversa, ya que el código era legible desde el primer momento.

7. Análisis dinámico

En esta fase pusimos a correr el binario dentro de un entorno controlado para observar cómo interactúa realmente con el sistema operativo y confirmar lo que sospechábamos en el análisis previo.

7.1 Comportamiento en ejecución

Al cargar el archivo en x64dbg, navegamos directamente al módulo principal para ignorar el código interno del sistema y enfocarnos solo en nuestro programa. Lo primero que notamos fue que el binario no hace nada de inmediato; se queda en una pausa larga. Al rastrear las llamadas, confirmamos que usa la función Sleep con un valor de 1388 en hexadecimal, lo que equivale a unos 5 segundos de espera. Esta es una técnica muy común para que el analista piense que el programa no funciona o para que las herramientas de análisis automático terminen su tiempo de revisión antes de que el "malware" actúe.

7.2 Intercepción de llamadas y registros

Pusimos puntos de interrupción (*breakpoints*) en las funciones clave para ver qué pasaba por la memoria justo en el momento del impacto:

- **WinExec:** Al detenernos aquí, revisamos el *stack* (la pila) y vimos claramente que el programa le estaba pasando la dirección de memoria donde está guardada la cadena "calc.exe" junto con el parámetro 5, que le ordena a Windows mostrar la ventana de la calculadora. En cuanto le dimos a "Step Over" (F8), la calculadora se abrió en la pantalla.
- **CreateFileA:** También interceptamos esta llamada y confirmamos que el binario estaba intentando crear el archivo dummyedu.txt en la ruta C:\temp\. Al revisar los registros después de la llamada, el valor en EAX nos confirmó que la operación fue exitosa y que el archivo se generó correctamente en el disco.

7.3 Monitoreo de memoria y procesos

Durante toda la ejecución, estuvimos pendientes de cómo cambiaban los registros. Vimos que el programa usa punteros para manejar las cadenas de texto, lo que nos permitió validar que la información que extrajimos en el análisis estático era exactamente la misma que se estaba procesando en tiempo real. No detectamos conexiones de red salientes ni intentos de inyectar código en otros procesos, lo que confirma que el comportamiento sospechoso está limitado únicamente a las acciones que programamos inicialmente.

8. Hallazgos (IOCs)

Después de darle vueltas al binario tanto por fuera como por dentro, logramos extraer varios Indicadores de Compromiso (IOCs) que sirven para identificar esta actividad sospechosa en cualquier sistema.

8.1 Identificadores del archivo

El primer punto de control son las firmas digitales del archivo. Aunque el nombre puede cambiar, estos hashes son únicos para esta versión del binario:

- **Hash SHA-256:** *(Aquí deberías pegar el hash que te dé el comando certutil -hashfile team_sample.exe SHA256 en tu terminal).*
- **Nombre original:** team_sample.exe.

8.2 Rastros en el sistema (Host-Based Indicators)

Estas son las huellas que deja el programa al interactuar con Windows:

- **Rutas de archivos:** Se detectó la creación persistente del archivo dummyedu.txt (o dummy.txt dependiendo de la compilación) dentro del directorio C:\temp\.
- **Procesos ejecutados:** El binario lanza de forma automática el proceso legítimo del sistema calc.exe desde su ubicación estándar en System32.

8.3 Firmas de memoria y comportamiento

Estos patrones nos ayudan a detectar el programa incluso si intenta esconderse:

- **Cadenas únicas:** La presencia de la cadena MAGIC: edu-torres-malware-sim en la memoria del proceso es el indicador más fuerte de que este binario específico está corriendo.
- **Patrón de API:** El uso combinado de Sleep (con un retardo de 5 segundos), WinExec y CreateFileA en un lapso corto de tiempo es una firma de comportamiento que categorizamos como sospechosa.

9. Diagramas de flujo y pseudocódigo

Para explicar cómo funciona el binario por dentro, desglosamos la lógica de la función principal. Esto nos ayuda a ver el orden de las acciones y cómo interactúa con el sistema operativo.

INICIO DEL PROGRAMA

// Paso 1: Identificación

IMPRIMIR en consola "MAGIC: edu-torres-malware-sim"

// Paso 2: Ejecución de proceso externo

LLAMAR a WinExec con el comando "calc.exe" y modo SW_SHOW

// Esto abre la calculadora de Windows visible para el usuario

// Paso 3: Técnica de evasión (retardo)

ESPERAR durante 5000 milisegundos (5 segundos)

// El programa se pausa para intentar saltarse análisis automáticos rápidos

// Paso 4: Manipulación de archivos

CREAR un archivo nuevo en la ruta "C:\temp\dummyedu.txt"

DAR permisos de escritura y configuración normal al archivo

// Paso 5: Finalización

IMPRIMIR mensaje de confirmación "Se ha ejecutado saludos..."

RETORNAR 0 y cerrar proceso

FIN DEL PROGRAMA

El flujo es lineal y directo, lo que facilitó mucho el análisis dinámico. Al no tener saltos condicionales complejos (if/else) o bucles pesados, pudimos rastrear cada API de Windows de forma secuencial en el debugger. El punto más crítico del flujo es el *Sleep*, ya que separa la fase de "presentación" (el mensaje en consola) de la fase de "impacto" (la creación del archivo).

10. Recomendaciones

Basándonos en lo que descubrimos al analizar este binario, estas son las acciones que sugerimos para proteger cualquier sistema contra archivos con comportamientos similares:

- **Controles Técnicos:** * Implementar la regla **YARA** que diseñamos para detectar la cadena "MAGIC" en tiempo real antes de que el proceso se ejecute por completo.

- Configurar políticas en el **EDR** para monitorear y alertar sobre cualquier proceso que intente usar WinExec para lanzar aplicaciones del sistema desde directorios temporales.
- Restringir los permisos de escritura en la carpeta C:\temp\ para usuarios estándar, evitando que binarios desconocidos puedan crear archivos de persistencia o logs.
- **Procedimientos Operativos:**
 - Mantener actualizadas las firmas de comportamiento en los sistemas de defensa para detectar técnicas de evasión como el uso de Sleep prolongados en binarios no firmados.
 - Realizar análisis de integridad periódicos en los directorios de sistema para asegurar que no existan archivos "dummy" que puedan ser usados para exfiltración o pruebas de penetración.

11. Conclusiones

El análisis completo de team_sample.exe nos permitió validar que incluso un binario sencillo puede esconder comportamientos sospechosos usando funciones estándar de Windows. Al cruzar los datos del análisis estático con lo que vimos en el debugger, confirmamos que el programa cumple exactamente con lo que dictaba su código fuente: se identifica, espera a que pase el tiempo de análisis automático y luego interactúa con el sistema.

En cuanto a la **fiabilidad de detección**, el uso de firmas estáticas (como los hashes y las cadenas de texto) es muy efectivo para este binario en particular. Sin embargo, el análisis dinámico en **x64dbg** fue lo que realmente nos dio la certeza de cómo opera, demostrando que ver los registros y la pila en tiempo real es indispensable para no dejarse engañar por las técnicas de evasión como los retardos.

Este ejercicio refuerza la importancia de tener una metodología bien estructurada para no pasar por alto detalles que, a simple vista, podrían parecer inofensivos.

12. Referencias y anexos

- **Material de Clase:** LSTI-FCFM - UANL. *F3 – Ingeniería inversa y análisis binario*, 2026.
- **Documentación de APIs:** Microsoft Learn. *WinExec function (winuser.h)* y *CreateFileA function (fileapi.h)*.
- **Manuales de Herramientas:**
 - NSA. *Ghidra Software Reverse Engineering Framework Documentation*.

- x64dbg. *Documentation and User Guide for Windows Debugging*.
- FireEye. *CAPA: Automatically identify capabilities in executable files*.

12.2 Anexos y registros técnicos

Para una revisión profunda de este análisis, se pueden consultar los siguientes archivos técnicos incluidos en la raíz del repositorio:

- **Logs de Debugging:** El archivo `debugging_log.md` contiene las capturas de pantalla de **x64dbg** donde se aprecian los registros EAX y el estado de la pila antes de cada llamada crítica.
- **Reporte de Capacidades:** El archivo `capa_report.txt` detalla todas las funcionalidades detectadas automáticamente, como la capacidad de crear archivos y ejecutar procesos.
- **Regla de Detección:** Se adjunta el archivo `team_rule.yar`, que contiene la lógica para identificar este binario basado en la cadena "MAGIC" y sus *imports*.
- **Binarios:** Se incluyen tanto el ejecutable original `team_sample.exe` como la versión modificada `team_sample_patched.exe` para pruebas comparativas.